

Corner Detection and Local Feature Descriptors

Harris Corner Detection, SIFT, FAST, BRIEF and ORB

Teaching Notes for Computer Vision

Contents

1	Introduction	4
2	Why Do We Need Corners?	4
3	Harris Corner Detection	5
4	The Sum of Squared Differences	6
5	Taylor Expansion	6
6	Matrix Form of the Harris Equation	7
7	Physical Meaning of the Structure Tensor	8
8	Connection with PCA	8
9	Why Do Eigenvalues Appear?	9
10	Physical Interpretation of Eigenvectors	9
11	Geometric Interpretation: The Error Ellipse	10
12	Computing the Eigenvalues	11
13	Harris Corner Response	12
14	Complete Harris Algorithm	13
15	From Corner Detection to Local Features	14
16	SIFT	14
17	Gaussian Scale Space	15
18	Difference of Gaussians	15
19	Scale-Space Extrema	15

20 Keypoint Localization	16
21 Rejecting Low-Contrast Keypoints	16
22 Rejecting Edge Responses	16
23 Orientation Assignment	17
24 SIFT Descriptor	18
25 SIFT Descriptor Construction	18
26 SIFT Normalization	19
27 SIFT Matching	19
28 FAST Corner Detector	19
29 The FAST Circle	20
30 FAST Segment Test	20
31 Why FAST is Fast	21
32 FAST Corner Response	21
33 FAST Non-Maximum Suppression	21
34 BRIEF Descriptor	22
35 BRIEF Binary Test	22
36 Why BRIEF is Computationally Efficient	22
37 Limitation of BRIEF: Rotation	23
38 ORB	23
39 ORB Keypoint Detection	24
40 Intensity Centroid for Orientation	24
41 Rotating the BRIEF Sampling Pattern	25
42 ORB and the Image Pyramid	25

43 ORB Descriptor	26
44 Why ORB is Fast	26
45 Comparison of the Feature Methods	26
46 Detailed Comparison	27
47 Descriptor Size and Matching Complexity	28
48 OpenCV Implementation	28
48.1 Harris Corner Detection	28
48.2 SIFT in OpenCV	29
48.3 FAST in OpenCV	29
48.4 BRIEF in OpenCV	30
48.5 ORB in OpenCV	30
49 Typical OpenCV Matching Pipeline	31
50 Feature Detection and Description: A Useful Taxonomy	31
51 Which Feature Should Be Used?	32
52 Overall Computational Perspective	33
53 Conceptual Summary	33
54 Final Comparison Table	34
55 Key Takeaways	35

1 Introduction

Edges are image locations where the intensity changes significantly in one direction.

However, an edge by itself is not necessarily a distinctive feature.

Consider a long straight edge. Along the direction of the edge, the image may look almost identical everywhere. Therefore, simply knowing that an edge exists does not tell us which particular location on the edge we are observing.

A corner is more distinctive because the image changes significantly in more than one direction.

This motivates the progression

$$\boxed{\text{Image} \rightarrow \text{Edges} \rightarrow \text{Corners} / \text{Interest Points} \rightarrow \text{Local Descriptors}} \quad (1)$$

A useful distinction is:

- **Feature detector:** determines *where* useful points are located.
- **Feature descriptor:** describes the local appearance around a detected point.
- **Feature matcher:** determines whether descriptors from two images correspond to each other.

For example:

$$\boxed{\text{Harris} / \text{FAST} \rightarrow \text{Feature Detection}} \quad (2)$$

whereas

$$\boxed{\text{SIFT} / \text{BRIEF} / \text{ORB} \rightarrow \text{Feature Description}} \quad (3)$$

Although SIFT and ORB contain both detection and description stages, the distinction is useful for understanding the individual components.

2 Why Do We Need Corners?

Consider three simple image regions.

1. Flat Region

For a flat region,

$$I(x, y) \approx C \quad (4)$$

for some constant C .

Therefore,

$$I_x \approx 0, \quad I_y \approx 0. \quad (5)$$

There is little intensity variation in any direction.

2. Edge

For an ideal vertical edge,

$$I(x, y) \approx f(x). \quad (6)$$

Thus,

$$I_x \neq 0, \quad I_y \approx 0. \quad (7)$$

The image changes strongly in one direction but not the other.

3. Corner

At a corner, the intensity changes significantly in two independent directions:

$$I_x \neq 0, \quad I_y \neq 0. \quad (8)$$

More importantly, the image should exhibit significant variation for motion in *any* direction.

This gives the basic classification:

Region	Intensity variation
Flat	Small in all directions
Edge	Large in one direction
Corner	Large in two directions

(9)

3 Harris Corner Detection

The Harris detector is based on a simple question:

How much does the image change if a small window is shifted?

Suppose an image window is centered at (x, y) .

We shift the window by

$$\mathbf{u} = \begin{bmatrix} u \\ v \end{bmatrix}. \quad (10)$$

If the image is locally flat, the shifted window should look similar.

If the window is centered on an edge, shifting along the edge produces little change.

If the window is centered on a corner, shifting in almost any direction produces a significant change.

This observation leads directly to the Harris formulation.

4 The Sum of Squared Differences

Let the image intensity be

$$I(x, y). \quad (11)$$

Consider a small window W around a point.

The difference between the original window and the shifted window is

$$I(x + u, y + v) - I(x, y). \quad (12)$$

We define the windowed sum of squared differences as

$$E(u, v) = \sum_{(x,y) \in W} w(x, y) [I(x + u, y + v) - I(x, y)]^2 \quad (13)$$

where $w(x, y)$ is a weighting function.

Usually, w is either:

- a box filter, or
- a Gaussian weighting function.

The Gaussian weighting is often preferable because pixels near the center of the window receive greater importance.

5 Taylor Expansion

For a sufficiently small displacement (u, v) , we approximate the shifted image using the first-order Taylor expansion:

$$I(x + u, y + v) \approx I(x, y) + I_x u + I_y v. \quad (14)$$

Therefore,

$$I(x + u, y + v) - I(x, y) \approx I_x u + I_y v. \quad (15)$$

Substituting this into the SSD expression gives

$$E(u, v) \approx \sum_{(x,y) \in W} w(x, y) (I_x u + I_y v)^2. \quad (16)$$

6 Matrix Form of the Harris Equation

Expand the square:

$$(I_x u + I_y v)^2 = I_x^2 u^2 + 2I_x I_y uv + I_y^2 v^2. \quad (17)$$

Therefore,

$$E(u, v) \approx \sum_W w(x, y) (I_x^2 u^2 + 2I_x I_y uv + I_y^2 v^2). \quad (18)$$

Since u and v do not depend on the summation index, we can rearrange:

$$E(u, v) \approx u^2 \sum_W w I_x^2 + 2uv \sum_W w I_x I_y + v^2 \sum_W w I_y^2. \quad (19)$$

Define

$$S_{xx} = \sum_W w I_x^2, \quad (20)$$

$$S_{xy} = \sum_W w I_x I_y, \quad (21)$$

and

$$S_{yy} = \sum_W w I_y^2. \quad (22)$$

Then

$$E(u, v) = S_{xx} u^2 + 2S_{xy} uv + S_{yy} v^2. \quad (23)$$

This can be written compactly as

$$E(\mathbf{u}) = \mathbf{u}^T \mathbf{M} \mathbf{u} \quad (24)$$

where

$$\mathbf{M} = \begin{bmatrix} S_{xx} & S_{xy} \\ S_{xy} & S_{yy} \end{bmatrix}. \quad (25)$$

The matrix $\mathbf{M}$ is called the

$$\boxed{\text{second-moment matrix}} \quad (26)$$

or

$$\boxed{\text{structure tensor.}} \quad (27)$$

7 Physical Meaning of the Structure Tensor

The structure tensor is

$$\mathbf{M} = \sum_W w \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}. \quad (28)$$

For an individual pixel, define the gradient

$$\mathbf{g} = \begin{bmatrix} I_x \\ I_y \end{bmatrix}. \quad (29)$$

Then

$$\mathbf{g}\mathbf{g}^T = \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}. \quad (30)$$

Therefore,

$$\boxed{\mathbf{M} = \sum_W w \mathbf{g}\mathbf{g}^T.} \quad (31)$$

This is important because it reveals the connection between Harris corner detection and covariance/PCA-type analysis.

8 Connection with PCA

PCA analyzes the dominant directions of variation in data.

Suppose we have a collection of vectors

$$\mathbf{g}_1, \mathbf{g}_2, \dots, \mathbf{g}_N. \quad (32)$$

A second-order matrix describing their directional variation is

$$\mathbf{C} = \sum_i w_i \mathbf{g}_i \mathbf{g}_i^T. \quad (33)$$

The Harris structure tensor has exactly this form:

$$\boxed{\mathbf{M} = \sum_i w_i \mathbf{g}_i \mathbf{g}_i^T.} \quad (34)$$

Thus, the eigenvectors of $\mathbf{M}$ identify the principal directions of local image variation.

The eigenvalues indicate the amount of variation in those directions.

This gives the key interpretation:

Eigenvectors $\rightarrow$ principal directions of local variation, Eigenvalues $\rightarrow$ strength of variation along those directions.	(35)
--	------

9 Why Do Eigenvalues Appear?

Recall the Harris energy:

$$E(\mathbf{u}) = \mathbf{u}^T \mathbf{M} \mathbf{u}. \quad (36)$$

Since $\mathbf{M}$ is symmetric, it has orthogonal eigenvectors.

Let

$$\mathbf{M} \mathbf{e}_1 = \lambda_1 \mathbf{e}_1 \quad (37)$$

and

$$\mathbf{M} \mathbf{e}_2 = \lambda_2 \mathbf{e}_2. \quad (38)$$

Because the eigenvectors form an orthonormal basis, any displacement can be expressed as

$$\mathbf{u} = \alpha \mathbf{e}_1 + \beta \mathbf{e}_2. \quad (39)$$

Substituting into the energy expression gives

$E = \lambda_1 \alpha^2 + \lambda_2 \beta^2.$

(40)

This equation is extremely important.

It tells us that the image-change energy along the principal direction $\mathbf{e}_1$ is controlled by λ_1 , while the energy along $\mathbf{e}_2$ is controlled by λ_2 .

Therefore,

$\lambda_i = \text{local image variation along eigenvector } \mathbf{e}_i.$

(41)

10 Physical Interpretation of Eigenvectors

The eigenvectors describe the directions in which the local image structure behaves most distinctly.

Suppose

$$\lambda_1 \gg \lambda_2. \quad (42)$$

Then there is strong variation in one direction but weak variation in the perpendicular direction.

This corresponds to an **edge**.

On the other hand, if

$$\lambda_1 \approx \lambda_2 \quad (43)$$

and both are large, then the image changes significantly in two independent directions.

This corresponds to a **corner**.

If

$$\lambda_1 \approx 0, \quad \lambda_2 \approx 0, \quad (44)$$

there is little variation in any direction.

This corresponds to a **flat region**.

Hence,

Region	λ_1	λ_2
Flat	≈ 0	≈ 0
Edge	large	≈ 0
Corner	large	large

(45)

assuming

$$\lambda_1 \geq \lambda_2. \quad (46)$$

11 Geometric Interpretation: The Error Ellipse

Consider

$$E(\mathbf{u}) = \mathbf{u}^T \mathbf{M} \mathbf{u}. \quad (47)$$

For a fixed energy value c ,

$$\mathbf{u}^T \mathbf{M} \mathbf{u} = c \quad (48)$$

defines an ellipse.

In the eigenvector coordinate system,

$$\lambda_1 \alpha^2 + \lambda_2 \beta^2 = c. \quad (49)$$

Rearranging,

$$\frac{\alpha^2}{c/\lambda_1} + \frac{\beta^2}{c/\lambda_2} = 1. \quad (50)$$

Therefore, the semi-axis lengths are

$$a_1 = \sqrt{\frac{c}{\lambda_1}} \quad (51)$$

and

$$a_2 = \sqrt{\frac{c}{\lambda_2}}. \quad (52)$$

For an edge,

$$\lambda_1 \gg \lambda_2, \quad (53)$$

so the ellipse becomes highly elongated.

For a corner,

$$\lambda_1 \approx \lambda_2 \gg 0, \quad (54)$$

so the ellipse is approximately circular.

Thus,

Flat $\rightarrow$ large ellipse,	Edge $\rightarrow$ elongated ellipse,	Corner $\rightarrow$ small approximately circular ellipse.
-----------------------------------	---------------------------------------	--

(55)

12 Computing the Eigenvalues

For

$$\mathbf{M} = \begin{bmatrix} S_{xx} & S_{xy} \\ S_{xy} & S_{yy} \end{bmatrix}, \quad (56)$$

the eigenvalues satisfy

$$\det(\mathbf{M} - \lambda \mathbf{I}) = 0. \quad (57)$$

Therefore,

$$\begin{vmatrix} S_{xx} - \lambda & S_{xy} \\ S_{xy} & S_{yy} - \lambda \end{vmatrix} = 0. \quad (58)$$

Expanding,

$$(S_{xx} - \lambda)(S_{yy} - \lambda) - S_{xy}^2 = 0. \quad (59)$$

Therefore,

$$\lambda^2 - (S_{xx} + S_{yy})\lambda + (S_{xx}S_{yy} - S_{xy}^2) = 0. \quad (60)$$

The eigenvalues are

$$\lambda_{1,2} = \frac{(S_{xx} + S_{yy}) \pm \sqrt{(S_{xx} - S_{yy})^2 + 4S_{xy}^2}}{2}. \quad (61)$$

13 Harris Corner Response

Computing eigenvalues explicitly at every pixel is possible, but Harris proposed a simpler response function.

Define

$$R = \det(\mathbf{M}) - k(\text{trace}(\mathbf{M}))^2 \quad (62)$$

where k is typically a small constant.

Since

$$\det(\mathbf{M}) = \lambda_1 \lambda_2 \quad (63)$$

and

$$\text{trace}(\mathbf{M}) = \lambda_1 + \lambda_2, \quad (64)$$

we obtain

$$R = \lambda_1 \lambda_2 - k(\lambda_1 + \lambda_2)^2. \quad (65)$$

This gives the following interpretation.

Flat Region

$$\lambda_1 \approx \lambda_2 \approx 0 \quad (66)$$

so

$$R \approx 0. \quad (67)$$

Edge

Suppose

$$\lambda_1 \gg 0, \quad \lambda_2 \approx 0. \quad (68)$$

Then

$$R \approx -k\lambda_1^2 < 0. \quad (69)$$

Corner

If

$$\lambda_1 \approx \lambda_2 \gg 0, \quad (70)$$

then

$$R > 0 \quad (71)$$

for an appropriate choice of k .

Thus,

R	Interpretation
$R \approx 0$	Flat region
$R < 0$	Edge
$R \gg 0$	Corner

(72)

14 Complete Harris Algorithm

The practical Harris pipeline is:

1. Compute image gradients I_x and I_y .

2. Compute

$$I_x^2, \quad I_y^2, \quad I_x I_y.$$

3. Smooth these quantities using a window or Gaussian filter.

4. Construct the structure tensor

$$\mathbf{M} = \begin{bmatrix} S_{xx} & S_{xy} \\ S_{xy} & S_{yy} \end{bmatrix}.$$

5. Calculate

$$R = \det(\mathbf{M}) - k(\text{trace}(\mathbf{M}))^2.$$

6. Threshold the response.

7. Apply non-maximum suppression to retain spatially distinct corners.

The complete pipeline is

$$\boxed{I \rightarrow (I_x, I_y) \rightarrow (I_x^2, I_y^2, I_x I_y) \rightarrow \mathbf{M} \rightarrow R \rightarrow \text{Threshold} \rightarrow \text{NMS} \rightarrow \text{Corners.}} \quad (73)$$

15 From Corner Detection to Local Features

A corner detector answers:

$$\boxed{\text{Where is an interesting point?}} \quad (74)$$

But many applications require another question:

$$\boxed{\text{What does the neighborhood around this point look like?}} \quad (75)$$

This is the role of a descriptor.

For a detected point p , we extract a local image patch

$$P_p. \quad (76)$$

A descriptor converts this patch into a vector:

$$\boxed{P_p \rightarrow \mathbf{d}_p.} \quad (77)$$

Two image points are considered potential matches when their descriptors are similar.

16 SIFT

SIFT stands for

Scale-Invariant Feature Transform.

SIFT was designed to produce features that are robust to:

- scale changes,
- image rotation,
- moderate illumination changes,
- moderate viewpoint changes,
- local image deformation.

A major idea is that the feature should not depend strongly on the particular resolution at which the image is observed.

The SIFT pipeline consists of:

$$\boxed{\begin{array}{l} \text{Scale-space construction} \\ \rightarrow \text{Extrema detection} \\ \rightarrow \text{Keypoint localization} \\ \rightarrow \text{Orientation assignment} \\ \rightarrow \text{Descriptor construction.} \end{array}} \quad (78)$$

17 Gaussian Scale Space

The Gaussian-smoothed image at scale σ is

$$\boxed{L(x, y, \sigma) = G(x, y, \sigma) * I(x, y)} \quad (79)$$

where

$$\boxed{G(x, y, \sigma) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)}. \quad (80)$$

Increasing σ produces increasingly blurred images.

Thus,

$$\sigma_1 < \sigma_2 \quad \Rightarrow \quad L(\sigma_2) \text{ is more blurred than } L(\sigma_1). \quad (81)$$

18 Difference of Gaussians

SIFT uses the Difference of Gaussians (DoG):

$$\boxed{D(x, y, \sigma) = L(x, y, k\sigma) - L(x, y, \sigma)}. \quad (82)$$

The DoG approximates the scale-normalized Laplacian of Gaussian:

$$\boxed{D(x, y, \sigma) \propto \sigma^2 \nabla^2 L(x, y, \sigma)}. \quad (83)$$

Therefore, strong extrema in the DoG correspond to candidate scale-space keypoints.

19 Scale-Space Extrema

A candidate point is compared against its neighbors in both spatial and scale dimensions.

For a point at location (x, y) and scale level σ , SIFT examines a $3 \times 3 \times 3$ neighborhood:

$$\boxed{3 \times 3 \text{ spatial neighborhood} \times 3 \text{ scale levels} = 27} \quad (84)$$

including the candidate itself.

Therefore, the candidate is compared with

$$26 \quad (85)$$

neighbors.

A point is retained if it is either greater than all neighbors or smaller than all neighbors.

This produces scale-space extrema.

20 Keypoint Localization

The discrete extrema are refined using a local Taylor expansion of the DoG function.
Let

$$\mathbf{X} = \begin{bmatrix} x \\ y \\ \sigma \end{bmatrix}. \quad (86)$$

Around a candidate location $\mathbf{X}_0$,

$$D(\mathbf{X}) \approx D + \nabla D^T \mathbf{X} + \frac{1}{2} \mathbf{X}^T H \mathbf{X} \quad (87)$$

where H is the Hessian matrix.

The stationary point satisfies

$$\nabla D(\mathbf{X}) = 0. \quad (88)$$

Thus,

$$\nabla D + H \hat{\mathbf{X}} = 0 \quad (89)$$

and therefore

$$\boxed{\hat{\mathbf{X}} = -H^{-1} \nabla D.} \quad (90)$$

This gives a sub-pixel and sub-scale estimate of the keypoint location.

21 Rejecting Low-Contrast Keypoints

Very weak extrema are unstable.

Therefore, SIFT rejects points whose interpolated DoG magnitude is too small.

Conceptually,

$$\boxed{|D(\hat{\mathbf{X}})| < T \quad \Rightarrow \quad \text{reject keypoint.}} \quad (91)$$

This prevents noisy, poorly localized points from becoming features.

22 Rejecting Edge Responses

A DoG extremum may be strong because it lies on an edge rather than at a well-localized keypoint.

SIFT uses the Hessian in the spatial directions:

$$H = \begin{bmatrix} D_{xx} & D_{xy} \\ D_{xy} & D_{yy} \end{bmatrix}. \quad (92)$$

Let the principal curvatures be proportional to the eigenvalues

$$\alpha, \beta. \quad (93)$$

The ratio of principal curvatures is used to identify edge-like responses.

Since

$$\text{Tr}(H) = \alpha + \beta \quad (94)$$

and

$$\det(H) = \alpha\beta, \quad (95)$$

SIFT uses a ratio test based on

$$\boxed{\frac{(\text{Tr } H)^2}{\det(H)}} \quad (96)$$

to reject highly anisotropic edge responses.

23 Orientation Assignment

To obtain rotation invariance, SIFT assigns a dominant orientation to each keypoint.

The image gradient is

$$m(x, y) = \sqrt{L_x^2 + L_y^2} \quad (97)$$

and the orientation is

$$\boxed{\theta(x, y) = \text{atan2}(L_y, L_x)}. \quad (98)$$

A weighted orientation histogram is constructed around the keypoint.

A Gaussian weighting function is commonly used:

$$w(x, y) = \exp\left(-\frac{x^2 + y^2}{2\sigma_w^2}\right). \quad (99)$$

Each sample contributes approximately

$$\boxed{w(x, y)m(x, y)} \quad (100)$$

to the histogram bin corresponding to its orientation.

The dominant peak determines the keypoint orientation.

24 SIFT Descriptor

After assigning orientation, the local neighborhood is rotated into the keypoint's canonical orientation.

A standard SIFT descriptor uses a

$$4 \times 4 \tag{101}$$

grid of spatial cells.

Each cell contains an orientation histogram with

$$8 \tag{102}$$

orientation bins.

Therefore, the descriptor dimensionality is

$$\boxed{4 \times 4 \times 8 = 128.} \tag{103}$$

Thus,

$$\boxed{\text{SIFT descriptor} \in \mathbb{R}^{128}.} \tag{104}$$

25 SIFT Descriptor Construction

For every pixel in the local neighborhood:

1. Compute gradient magnitude.
2. Compute gradient orientation.
3. Rotate the coordinate system according to the keypoint orientation.
4. Determine the spatial cell.
5. Determine the orientation histogram bin.
6. Weight the contribution using gradient magnitude and spatial weighting.

The resulting descriptor is

$$\mathbf{d} = [d_1, d_2, \dots, d_{128}]^T. \tag{105}$$

The descriptor is normalized to reduce sensitivity to illumination changes.

26 SIFT Normalization

Let

$$\mathbf{d} = [d_1, \dots, d_{128}]^T. \quad (106)$$

The first normalization is

$$\boxed{\mathbf{d} \leftarrow \frac{\mathbf{d}}{\|\mathbf{d}\|_2}.} \quad (107)$$

Large values are then thresholded, typically at a fraction of the maximum normalized value:

$$d_i \leftarrow \min(d_i, 0.2). \quad (108)$$

Finally, the descriptor is renormalized.

This reduces the influence of unusually strong gradient responses.

27 SIFT Matching

Because SIFT descriptors are floating-point vectors, Euclidean distance is commonly used:

$$\boxed{d(\mathbf{d}_1, \mathbf{d}_2) = \sqrt{\sum_{i=1}^{128} (d_{1i} - d_{2i})^2}.} \quad (109)$$

A common matching strategy is nearest-neighbor matching.

For a descriptor $\mathbf{d}_q$ in image 1,

$$j^* = \arg \min_j d(\mathbf{d}_q, \mathbf{d}_j). \quad (110)$$

The ratio test compares the best and second-best matches:

$$\boxed{\frac{d_1}{d_2} < \tau.} \quad (111)$$

This helps reject ambiguous matches.

28 FAST Corner Detector

FAST stands for

Features from Accelerated Segment Test.

FAST is designed primarily for very rapid interest-point detection.

Instead of calculating image derivatives and eigenvalues, FAST asks a simpler question:

Are there enough pixels on a circle around this pixel that are significantly brighter or darker than the center?

29 The FAST Circle

For a candidate pixel p with intensity

$$I_p, \tag{112}$$

FAST considers a circle of 16 pixels around it.

Let the intensity of circle pixel x be

$$I_x. \tag{113}$$

Choose a threshold t .

A pixel belongs to the bright set if

$$\boxed{I_x \geq I_p + t} \tag{114}$$

and to the dark set if

$$\boxed{I_x \leq I_p - t.} \tag{115}$$

Otherwise,

$$I_p - t < I_x < I_p + t. \tag{116}$$

30 FAST Segment Test

A candidate pixel is classified as a corner if there exists a contiguous sequence of N pixels on the circle satisfying either the bright or dark condition.

For FAST-9,

$$N = 9. \tag{117}$$

For example, the bright condition is

$$\boxed{I_{x_j} \geq I_p + t} \tag{118}$$

for N contiguous pixels.

Similarly, the dark condition is

$$\boxed{I_{x_j} \leq I_p - t.} \quad (119)$$

Thus,

$$\boxed{\text{FAST corner} \iff \text{long contiguous bright or dark arc.}} \quad (120)$$

31 Why FAST is Fast

Testing all 16 pixels for every image pixel would still be unnecessarily expensive.

FAST uses an early rejection strategy.

A small number of strategically chosen pixels are examined first.

If these pixels cannot possibly satisfy the segment condition, the candidate is rejected immediately.

Thus,

$$\boxed{\text{Early rejection} \rightarrow \text{far fewer intensity comparisons.}} \quad (121)$$

This is one of the main reasons FAST is computationally efficient.

The decision can also be represented as a decision tree learned from training images.

32 FAST Corner Response

The basic FAST segment test only determines whether a point is a corner.

It does not naturally produce a strong corner response for ranking points.

A score can therefore be associated with a candidate.

One useful conceptual score is the largest threshold for which the point continues to satisfy the segment test:

$$\boxed{S(p) = \max\{t : p \text{ satisfies the FAST criterion at threshold } t\}.} \quad (122)$$

The exact scoring and implementation details depend on the FAST variant and library implementation.

33 FAST Non-Maximum Suppression

Nearby pixels may all satisfy the FAST condition.

Therefore, non-maximum suppression is applied.

For a neighborhood $\mathcal{N}(p)$,

$$\boxed{p \text{ is retained if } S(p) \geq S(q) \quad \forall q \in \mathcal{N}(p).} \quad (123)$$

This produces a sparse set of strong interest points.

34 BRIEF Descriptor

BRIEF stands for

Binary Robust Independent Elementary Features.

BRIEF is a descriptor rather than a complete scale- and rotation-invariant feature detector.

Its central idea is extremely simple:

Compare the intensities of pairs of pixels in a local patch.

35 BRIEF Binary Test

Let a smoothed image patch be denoted by P .

Choose two locations

$$\mathbf{x}_i \quad \text{and} \quad \mathbf{y}_i. \quad (124)$$

Define a binary intensity comparison:

$$\tau(P; \mathbf{x}_i, \mathbf{y}_i) = \begin{cases} 1, & P(\mathbf{x}_i) < P(\mathbf{y}_i), \\ 0, & P(\mathbf{x}_i) \geq P(\mathbf{y}_i). \end{cases} \quad (125)$$

Repeat this for n pairs.

The descriptor becomes

$$\text{BRIEF}(P) = [\tau_1, \tau_2, \dots, \tau_n]. \quad (126)$$

For example, with 256 tests,

$$\text{BRIEF}(P) \in \{0, 1\}^{256}. \quad (127)$$

36 Why BRIEF is Computationally Efficient

A floating-point descriptor requires operations such as:

$$(x_1 - x_2)^2 \quad (128)$$

and

$$\sum_i (x_i - y_i)^2. \quad (129)$$

BRIEF instead uses binary comparisons.

Therefore, matching can use the Hamming distance.

For binary descriptors

$$\mathbf{b}_1, \mathbf{b}_2, \quad (130)$$

the Hamming distance is

$$d_H(\mathbf{b}_1, \mathbf{b}_2) = \sum_i b_{1i} \oplus b_{2i}. \quad (131)$$

In hardware/software this can be implemented efficiently using bitwise XOR followed by a population count.

Thus,

$$\boxed{\text{Binary descriptor} \rightarrow \text{XOR} \rightarrow \text{bit count.}} \quad (132)$$

This is much cheaper than floating-point vector distance.

37 Limitation of BRIEF: Rotation

Consider a patch containing an edge.

If the image rotates, the spatial relationship between the sampled points changes.

The same binary tests may therefore produce completely different results.

Hence, basic BRIEF is not rotation invariant.

Similarly, because its sampling pattern has a fixed scale, BRIEF does not inherently provide scale invariance.

Thus,

$$\boxed{\text{BRIEF} \rightarrow \text{fast descriptor} \quad \text{but not inherently rotation/scale invariant.}} \quad (133)$$

This limitation motivates ORB.

38 ORB

ORB stands for

Oriented **F**AST and **R**otated **B**RIEF.

ORB combines:

$$\boxed{\text{FAST} + \text{Orientation} + \text{Rotated BRIEF.}} \quad (134)$$

The overall idea is:

$$\boxed{\text{FAST} \rightarrow \text{Keypoints} \rightarrow \text{Orientation} \rightarrow \text{Rotated BRIEF} \rightarrow \text{Binary Descriptor}.} \quad (135)$$

ORB additionally uses an image pyramid to provide some degree of scale robustness.

39 ORB Keypoint Detection

ORB uses FAST to detect candidate keypoints.

However, simply detecting FAST corners does not provide a reliable orientation.

Therefore, ORB estimates the orientation using the intensity distribution around the keypoint.

40 Intensity Centroid for Orientation

Consider a local patch around a keypoint.

Define its image moments:

$$\boxed{m_{pq} = \sum_x \sum_y x^p y^q I(x, y).} \quad (136)$$

The zeroth-order moment is

$$m_{00} = \sum_x \sum_y I(x, y). \quad (137)$$

The first-order moments are

$$m_{10} = \sum_x \sum_y x I(x, y) \quad (138)$$

and

$$m_{01} = \sum_x \sum_y y I(x, y). \quad (139)$$

The intensity centroid is therefore

$$\boxed{C_x = \frac{m_{10}}{m_{00}}, \quad C_y = \frac{m_{01}}{m_{00}}.} \quad (140)$$

The vector from the keypoint to the intensity centroid is

$$\mathbf{c} = \begin{bmatrix} C_x \\ C_y \end{bmatrix}. \quad (141)$$

The orientation is

$$\theta = \text{atan2}(C_y, C_x). \quad (142)$$

This gives ORB an orientation estimate.

41 Rotating the BRIEF Sampling Pattern

Suppose the original BRIEF sampling locations are

$$\mathbf{x}_i \quad \text{and} \quad \mathbf{y}_i. \quad (143)$$

The 2D rotation matrix is

$$R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}. \quad (144)$$

The sampling points are rotated as

$$\mathbf{x}'_i = R(\theta)\mathbf{x}_i \quad (145)$$

and

$$\mathbf{y}'_i = R(\theta)\mathbf{y}_i. \quad (146)$$

The binary test then becomes

$$\tau_i = \tau(P; \mathbf{x}'_i, \mathbf{y}'_i). \quad (147)$$

Thus, when the image rotates, the sampling pattern rotates with the local feature.

This produces approximate rotation invariance.

42 ORB and the Image Pyramid

ORB uses multiple image scales.

Let

$$I_0, I_1, \dots, I_L \quad (148)$$

denote the pyramid levels.

Each level represents a different effective image scale.

FAST detection is performed across the pyramid.

Therefore, the same physical structure can potentially be detected at different image resolutions.

This gives ORB some degree of scale robustness.

It is important to note that this is not identical to the continuous scale-space treatment of SIFT.

43 ORB Descriptor

ORB uses a modified version of BRIEF called

$$\boxed{\text{rBRIEF}} \quad (149)$$

or rotated BRIEF.

The descriptor is still binary:

$$\boxed{\mathbf{d} \in \{0, 1\}^N.} \quad (150)$$

A commonly used ORB descriptor contains

$$256 \quad (151)$$

binary tests, corresponding to

$$\boxed{256 \text{ bits} = 32 \text{ bytes}.} \quad (152)$$

The descriptor is therefore very compact.

44 Why ORB is Fast

ORB combines several computationally efficient ideas:

1. FAST avoids expensive derivative-based corner detection.
2. Image pyramids provide scale handling.
3. Orientation is computed using simple image moments.
4. BRIEF uses binary intensity comparisons.
5. Binary descriptors are matched using Hamming distance.

Therefore,

$$\boxed{\text{FAST} + \text{binary descriptor} + \text{Hamming matching} \rightarrow \text{high computational efficiency}.} \quad (153)$$

45 Comparison of the Feature Methods

It is important to distinguish the roles of the methods.

Method	Primary Role	Main Mathematical Idea	Main Invariance / Property
Harris	Corner detector	Structure tensor and eigenvalue analysis	Good localization; not inherently scale invariant
SIFT	Detector + descriptor	Scale-space DoG + gradient histograms	Scale and rotation robustness
FAST	Corner detector	Contiguous intensity segment test	Very fast; limited invariance by itself
BRIEF	Descriptor	Binary intensity comparisons	Very compact and fast; no inherent rotation invariance
ORB	Detector + descriptor	FAST + intensity centroid + rotated BRIEF	Rotation robustness and practical scale robustness

46 Detailed Comparison

Feature	Type	Pros	Cons	Typical Applications	Applicability	Computational Load
Harris	Corner detector	Strong mathematical foundation; good localization; interpretable	Not inherently scale invariant; sensitive to large viewpoint and scale changes	Classical corner detection; tracking; optical flow initialization; image registration		Low–moderate
SIFT	Detector + descriptor	Strong scale and rotation robustness; highly distinctive; relatively robust to illumination and viewpoint changes	Computationally expensive; larger descriptor; floating-point matching	Image matching; panorama creation; image registration; object recognition; 3D reconstruction; SfM		High
FAST	Corner detector	Extremely fast; suitable for real-time systems; simple intensity comparisons	Not inherently scale or rotation invariant; detector only	Real-time tracking; SLAM; embedded vision; robotics; high-frame-rate applications		Very low
BRIEF	Descriptor	Very compact; binary; extremely fast matching	Not inherently rotation invariant; limited scale robustness; requires a detector	Fast image matching; tracking; resource-constrained systems		Very low
ORB	Detector + descriptor	Fast; binary descriptor; rotation robustness; practical scale robustness; low memory usage	Generally less distinctive than SIFT under difficult transformations; less robust to large viewpoint and scale changes	Visual odometry; SLAM; panorama stitching; real-time tracking; robotics; embedded vision		Low

47 Descriptor Size and Matching Complexity

A major difference between SIFT and binary descriptors is the representation.

SIFT:

$$\mathbf{d}_{SIFT} \in \mathbb{R}^{128}. \quad (154)$$

ORB:

$$\mathbf{d}_{ORB} \in \{0, 1\}^{256}. \quad (155)$$

SIFT matching typically uses Euclidean distance:

$$d_2 = \sqrt{\sum_i (d_i^{(1)} - d_i^{(2)})^2}. \quad (156)$$

ORB matching uses Hamming distance:

$$d_H = \text{popcount}(\mathbf{d}_1 \oplus \mathbf{d}_2). \quad (157)$$

Therefore,

$$\boxed{\text{Binary descriptor matching} \ll \text{floating-point descriptor matching}} \quad (158)$$

in computational cost in many practical implementations.

48 OpenCV Implementation

OpenCV provides implementations of all of these techniques, although the API structure differs between methods.

48.1 Harris Corner Detection

The classical OpenCV function is

$$\boxed{\text{cv2.cornerHarris()}} \quad (159)$$

A typical call is conceptually:

```
dst = cv2.cornerHarris(gray, blockSize, ksize, k)
```

where:

- **blockSize** is the neighborhood size used for calculating the local covariance/structure tensor.

- `ksize` is the Sobel derivative aperture.
- `k` is the Harris detector constant.

The response image contains a score for every pixel.

Thresholding and non-maximum suppression are generally required to obtain a clean set of corner coordinates.

48.2 SIFT in OpenCV

SIFT is available through

```
sift = cv2.SIFT_create()
keypoints, descriptors = sift.detectAndCompute(gray, None)
```

The result consists of:

- a list of `KeyPoint` objects,
- a floating-point descriptor matrix.

For N keypoints,

$$\boxed{\text{descriptors.shape} = (N, 128)} \quad (160)$$

for the standard SIFT descriptor.

SIFT descriptors are commonly matched using

```
cv2.NORM_L2
```

because they are floating-point vectors.

48.3 FAST in OpenCV

FAST is available through

```
fast = cv2.FastFeatureDetector_create()
keypoints = fast.detect(gray, None)
```

Important parameters include:

- intensity threshold,
- FAST type,
- non-maximum suppression.

FAST returns keypoints, but not descriptors.

Therefore,

$$\boxed{\text{FAST} \neq \text{complete feature matching pipeline.}} \quad (161)$$

A descriptor such as BRIEF or ORB must be added if matching is required.

48.4 BRIEF in OpenCV

BRIEF is historically available in OpenCV's feature modules, notably through the `xfeatures2d` module in installations where the corresponding contrib functionality is available.

Conceptually:

```
brief = cv2.xfeatures2d.BriefDescriptorExtractor_create()
keypoints, descriptors = brief.compute(gray, keypoints)
```

The important conceptual point is that BRIEF does not necessarily detect keypoints itself.

A detector must first provide keypoints:

$$\boxed{\text{Detector} \rightarrow \text{Keypoints} \rightarrow \text{BRIEF} \rightarrow \text{Descriptors.}} \quad (162)$$

Availability and API details can vary with the OpenCV version and whether the contrib modules are installed.

48.5 ORB in OpenCV

ORB is available directly through

```
orb = cv2.ORB_create()
keypoints, descriptors = orb.detectAndCompute(gray, None)
```

ORB returns both:

- keypoints,
- binary descriptors.

The descriptor is typically represented as an unsigned 8-bit array.

For N keypoints,

$$\boxed{\text{descriptors.shape} = (N, 32)} \quad (163)$$

because

$$32 \times 8 = 256 \quad (164)$$

bits.

ORB descriptors should therefore be matched using Hamming distance:

`cv2.NORM_HAMMING`

rather than Euclidean distance.

49 Typical OpenCV Matching Pipeline

For SIFT:

```
sift = cv2.SIFT_create()

kp1, des1 = sift.detectAndCompute(img1, None)
kp2, des2 = sift.detectAndCompute(img2, None)

bf = cv2.BFMatcher(cv2.NORM_L2)
matches = bf.knnMatch(des1, des2, k=2)
```

For ORB:

```
orb = cv2.ORB_create()

kp1, des1 = orb.detectAndCompute(img1, None)
kp2, des2 = orb.detectAndCompute(img2, None)

bf = cv2.BFMatcher(cv2.NORM_HAMMING)
matches = bf.knnMatch(des1, des2, k=2)
```

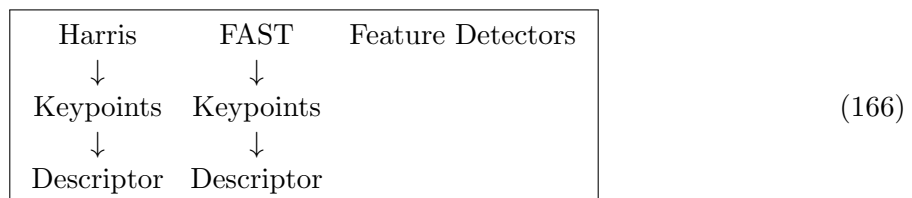
The important difference is

SIFT $\rightarrow$ L2 distance, ORB/BRIEF $\rightarrow$ Hamming distance.
--

(165)

50 Feature Detection and Description: A Useful Taxonomy

The methods can be arranged according to their roles.



SIFT combines detection and description:

SIFT = Scale-space detector + Gradient descriptor.
--

(167)

ORB similarly combines detection and description:

$$\boxed{\text{ORB} = \text{FAST detector} + \text{Orientation} + \text{Binary descriptor.}} \quad (168)$$

BRIEF is primarily a descriptor:

$$\boxed{\text{BRIEF} = \text{Binary local descriptor.}} \quad (169)$$

51 Which Feature Should Be Used?

There is no universally best feature.

The appropriate choice depends on the application.

Classical Image Registration

If robustness is more important than computational cost,

$$\boxed{\text{SIFT}} \quad (170)$$

is a strong classical choice.

Real-Time Robotics

For systems where computational resources are constrained,

$$\boxed{\text{ORB}} \quad (171)$$

is often attractive.

Very Fast Interest-Point Detection

If only keypoint detection is required,

$$\boxed{\text{FAST}} \quad (172)$$

is highly efficient.

Simple Classical Corner Detection

For teaching and mathematical understanding,

$$\boxed{\text{Harris}} \quad (173)$$

is particularly useful because the connection to gradients, quadratic forms, eigenvalues, PCA, and local image geometry is very clear.

Extremely Lightweight Binary Description

If a detector is already available and a compact descriptor is required,

$$\boxed{\text{BRIEF}} \quad (174)$$

is conceptually simple and computationally inexpensive.

52 Overall Computational Perspective

A rough conceptual ordering from computationally expensive to computationally inexpensive is

$$\boxed{\text{SIFT} > \text{Harris} > \text{ORB} > \text{FAST/BRIEF}} \quad (175)$$

although actual runtime depends strongly on:

- image resolution,
- number of keypoints,
- pyramid levels,
- implementation,
- hardware,
- number of descriptors being matched.

The important qualitative distinction is:

$$\begin{array}{l} \text{SIFT} \rightarrow \text{high robustness, high computation,} \\ \text{ORB} \rightarrow \text{good robustness, low computation,} \\ \text{FAST} \rightarrow \text{very fast detection,} \\ \text{BRIEF} \rightarrow \text{very fast binary description.} \end{array} \quad (176)$$

53 Conceptual Summary

The progression from Harris to ORB can be understood as a progression in the question being asked.

Harris

$$\boxed{\text{Does the local image change significantly in two directions?}} \quad (177)$$

Mathematically,

$$\mathbf{M} = \sum_w w \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}. \quad (178)$$

Its eigenvalues describe local variation.

SIFT

Where is a distinctive structure across scale, and what is its local gradient pattern?

 (179)

It uses:

DoG scale space + gradient orientation histograms.

 (180)

FAST

Does a contiguous arc of pixels differ strongly from this candidate pixel?

 (181)

BRIEF

Which of two sampled pixels is brighter?

 (182)

Repeated many times, these binary tests create a compact descriptor.

ORB

Can we combine FAST detection with an oriented binary descriptor?

 (183)

The result is

ORB = FAST + Intensity Centroid Orientation + Rotated BRIEF.

 (184)

54 Final Comparison Table

Method	Output	Mathematical Core	Strengths	Weaknesses	Best-Suited Applications
Harris	Corners	Structure tensor; eigenvalues; Harris response	Interpretable; good localization; mathematically elegant	Scale sensitive; limited invariance	Corner localization, tracking initialization, teaching classical feature detection
SIFT	Keypoints + 128-D descriptors	DoG scale space; gradient histograms	Highly distinctive; scale and rotation robust	Relatively expensive; large floating-point descriptors	Image matching, SfM, registration, panorama, object recognition
FAST	Corners	16-pixel circle segment test	Extremely fast; simple	No descriptor; limited invariance	Real-time detection, robotics, tracking, embedded systems
BRIEF	Binary descriptor	Random/learned intensity comparisons	Very compact; extremely fast matching	No inherent rotation invariance; detector required	Lightweight matching and tracking

Method	Output	Mathematical Core	Strengths	Weaknesses	Best-Suited Applications
ORB	Keypoints + binary descriptors	FAST + intensity centroid + rBRIEF + pyramid	Fast; compact; rotation robust; practical scale robustness	Less robust than SIFT for difficult viewpoint/scale changes	SLAM, visual odometry, robotics, real-time matching

55 Key Takeaways

1. A **corner** is a location where image intensity changes significantly in two independent directions.
2. Harris represents this behavior using the structure tensor

$$\mathbf{M} = \sum_w w \mathbf{g} \mathbf{g}^T.$$

3. The eigenvectors of $\mathbf{M}$ represent the principal directions of local image variation.
4. The eigenvalues represent the amount of variation along those directions.
5. The Harris response

$$R = \det(M) - k(\text{trace } M)^2$$

avoids explicitly computing the eigenvalues.

6. SIFT obtains scale robustness through scale-space extrema and rotation robustness through dominant gradient orientation.
7. SIFT represents a local neighborhood using a 128-dimensional floating-point gradient histogram descriptor.
8. FAST obtains extremely fast corner detection by testing contiguous intensity segments on a circle around a candidate pixel.
9. BRIEF represents a patch using binary intensity comparisons.
10. ORB combines FAST with orientation estimation and rotated BRIEF to obtain a fast binary feature suitable for real-time applications.
11. SIFT generally provides stronger robustness and distinctiveness, while ORB provides a much better speed–memory–robustness trade-off for many real-time systems.

The overall conceptual progression is therefore

Harris : Local image geometry

↓

SIFT : Scale-space + gradient description

↓

FAST : Efficient interest-point detection

↓

BRIEF : Efficient binary description

↓

ORB : FAST + oriented binary description.

(185)